Programação Estruturada em C

Aula 4 — Consolidação de repetição, laços aninhados, break e continue

Prof. Me. Eng. Matheus Kotaki

Qual laço comunica melhor a intenção?

Quantidade / progressão

for

Útil quando existe uma progressão clara ou uma quantidade conhecida de repetições.

Condição

while

Natural quando repetimos enquanto uma condição permanecer verdadeira e não sabemos quantas vezes serão necessárias.

Pelo menos uma vez

do-while

Expressivo quando a ação precisa ocorrer pelo menos uma vez antes de decidir se será repetida.

Validação de entrada com scanf

scanf também informa se conseguiu ler

```
int idade;
int resultado;

printf("Idade: ");
resultado = scanf("%d", &idade);

if (resultado == 1) {
    printf("Idade lida corretamente");
} else {
    printf("Entrada invalida.\n");
}
```

scanf devolve um valor.

O scanf retorna a quantidade de valores que conseguiu ler e converter conforme os especificadores informados. Com %d, retorno 1 indica conversão realizada. Retorno 0 indica que não pôde ser convertido para inteiro.

Por que repetir o mesmo scanf pode falhar novamente?

```
int idade;
int resultado;

do {
    printf("Idade: ");
    resultado = scanf("%d", &idade);

    if (resultado != 1) {
        printf("Entrada invalida.\n");
    }

} while (resultado != 1);

printf("Idade lida corretamente: %d\n", idade);
```

Teste com: abc

scanf não consegue converter abc para int.

O conteúdo inválido permanece pendente no buffer de entrada.

A próxima tentativa encontra o mesmo conteúdo.

Laço infinito

Validação repetida com do-while

```
int idade;
int resultado;
int c;

do {
    printf("Idade: ");
    resultado = scanf("%d", &idade);

    if (resultado != 1) {
        printf("Entrada invalida.\n");

        while ((c = getchar()) != '\n' && c != EOF);
    }

} while (resultado != 1);

printf("Idade lida corretamente: %d\n", idade);
```

Tentar ler → verificar o retorno → **tratar a falha (limpa o buffer)** → repetir quando necessário.

Laços aninhados

Quando uma repetição acontece dentro de outra

Para 3 alunos, ler 2 notas de cada aluno e calcular a média de cada um.

Para cada aluno → **para cada nota desse aluno**

Duas responsabilidades de repetição → dois níveis de controle.

Laços aninhados

```
float soma;
float valor;

for (int aluno = 1; aluno <= 3; aluno++) {
    soma = 0;

    for (int nota = 1; nota <= 2; nota++) {
        printf("Digite a nota %d do Aluno %d:
", nota, aluno);
        scanf("%f", &valor);
        soma += valor;
    }

    printf("Media do Aluno %d: %.1f\n", aluno,
soma / 2);
}
```

soma = 0 fica dentro do laço externo.

Assim, cada aluno começa com seu próprio acumulador.

O for interno reinicia nota automaticamente a cada nova entrada nele.

Rastreando dois laços

```
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 3;
j++) {
        printf("%d %d\n", i,
j);
    }
}
```

Execução	i	j
1	1	1
2	1	2
3	1	3
4	2	1
5	2	2
6	2	3

Observe a passagem (1, 3) → (2, 1).

Um padrão simples para enxergar o limite variável

```
for (int linha = 1; linha <= 4; linha++)
{
    for (int coluna = 1; coluna <= linha;
coluna++) {
        printf("*");
    }
    printf("\n");
}
```

```
*
**
***
****
```

O interno executa 1, depois 2, depois 3 e depois 4 vezes.

Controle do fluxo

break: parar antes do limite quando não faz sentido continuar

```
int valor;

for (int tentativa = 1; tentativa <= 5;
tentativa++) {
    scanf("%d", &valor);

    if (valor == 7) {
        printf("Acertou!\n");
        break;
    }
}
```

break encerra o laço em que está inserido.

O limite permite até 5 tentativas, mas um acerto torna as próximas desnecessárias.

Em laços aninhados, break encerra apenas o mais interno

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 5; j++) {
        if (j == 3) {
            break;
        }
        printf("%d %d\n", i, j);
    }
}
```

Para cada i:

(i, 1)
(i, 2)
→ break quando j = 3
→ o próximo i ainda acontece

break não encerra automaticamente os dois laços.

continue: abandonar apenas a iteração atual

```
int soma = 0;
int valor = 0;

for (int i = 1; i <= 5; i++) {
    scanf("%d", &valor);

    if (valor <= 0) {
        continue;
    }
    printf("Somou\n");
    soma += valor;
}
```

continue não encerra o laço.

Ele interrompe o restante da iteração atual e deixa o fluxo seguir para a próxima etapa do laço.

Valores não positivos são ignorados na soma.

Cuidado: continue pode pular uma atualização necessária

```
int i = 0;

while (i < 10) {
    if (i == 5) {
        continue;
    }

    printf("%d\n", i);
    i++;
}
```

Quando i chega a 5:

- continue pula i++;
- i continua valendo 5;
- a condição continua verdadeira;
- o laço fica preso nesse estado.

Resultado: laço infinito.

Loop Infinito

```
char resp = 's';

while (1) {  // ou for(;;)
    printf("\nPreso no Loop\n");
    printf("Deseja continuar no
Loop?\n");
    scanf(" %c", &resp);
}
```

Laço nunca se quebra

É necessário adicionar o break

Prática 1

Desenvolva um programa que processe os resultados de 4 participantes. Cada participante realiza 3 tentativas. Para cada participante, o programa deve ler os resultados das 3 tentativas e informar qual foi o seu melhor resultado. Considere que um participante atingiu a meta quando seu melhor resultado for maior ou igual a 8,0. Ao final do programa, informe quantos participantes atingiram a meta.

Planeje antes de codificar.

Possível esqueleto do programa

```
for (int participante = 1; participante <= 4; participante++) {
    // reiniciar o estado deste participante
    ...

    for (int tentativa = 1; tentativa <= 3; tentativa++) {
        // ler e comparar o resultado
        ...
    }

    // apresentar o melhor e atualizar a contagem da meta
    ...
}

// apresentar a contagem da meta
...
```